🧪 Actividad obligatoria 2

Simulación con FastAPI de CoAP, WebSocket y MQTT usando HiveMQ

🎯 Objetivo de la actividad

El propósito es que el estudiante:

1. Ejecute simulaciones IoT con **FastAPI** usando:
    - **CoAP**
    - **WebSocket**
    - **MQTT**
2. Utilice un **bróker público (HiveMQ)** en lugar de uno local.
3. Se **suscriba desde un cliente web de HiveMQ** para visualizar los mensajes publicados.
4. Comprenda:
    - Diferencias entre protocolos en **tiempo real**.
    - Cómo cambia la **arquitectura cuando el bróker está en Internet**.
    - Qué ocurre en cada **capa del modelo de red**.

1️⃣ Preparación del entorno

Se parte del **repositorio ya forkeado y clonado** en la Actividad 1.

Entrar al proyecto:

```
cd iot_repo
```

Instalar dependencias (si aún no están):

```
pip install -r requirements.txt
```

o usando `uv`:

```
pip install uv
```

2️⃣ Simulación con FastAPI

FastAPI actuará como **servidor IoT** que:

- Recibe datos
- Publica eventos
- Permite comunicación en tiempo real

Para iniciar el servidor:

```
uv run uvicorn lectura:app --port 8000 --reload
```

Esto habilita:

- **HTTP**
- **WebSocket**
- Integraciones con **MQTT**
- Posible gateway hacia **CoAP**

3️⃣ Uso de MQTT con bróker público HiveMQ

A diferencia de la Actividad 1:

- **Ya no se usa broker local**
- Se usa **HiveMQ público en Internet**

3.1 Datos del broker HiveMQ público

Parámetros típicos:

- **Host:** `broker.hivemq.com`
- **Puerto sin TLS:** `1883`
- **Puerto con TLS:** `8883`
- **WebSocket:** `8000` o `8884` según endpoint web

3.2 Ejecutar el publicador MQTT

En otra terminal:

```
uv run python sensor_mq.py
```

Este script:

- Genera datos de sensor simulados
- Publica en un **topic MQTT**
- Envía los mensajes al **broker público**

3.3 Ejecutar el suscriptor local (opcional)

```
uv run python lectura_mq.py
```

Permite ver:

- Recepción local de mensajes
- Confirmación de publicación correcta

4️⃣ Suscripción desde cliente web de HiveMQ

Ahora viene la parte clave de esta actividad:

👉 **Ver los mensajes desde Internet**

Pasos:

1. Abrir el cliente web oficial:

   HiveMQ WebSocket Client

2. Configurar conexión:
   - **Host:** `broker.hivemq.com`
   - **Puerto:** `8000` (WebSocket)
   - **Path:** `/mqtt`
   - **Client ID:** cualquiera único
3. Conectarse.
4. En la sección **Subscriptions**:
   - Escribir el mismo **topic** usado por `sensor_mq.py`.
   - Presionar **Subscribe**.
5. Observar:
   - Llegada de mensajes en tiempo real.
   - Contenido del payload enviado desde tu PC.

5️⃣ Simulación con WebSocket usando FastAPI

FastAPI permite comunicación **bidireccional en tiempo real**.

Concepto clave

- HTTP → petición/respuesta
- **WebSocket → conexión persistente**
- Permite:
    - Streaming de datos
    - Notificaciones en vivo
    - Dashboards IoT

Prueba típica

1. Ejecutar FastAPI.
2. Conectar un cliente WebSocket (navegador o script).
3. Enviar/recibir mensajes en tiempo real.
4. Comparar con MQTT:

Característica	WebSocket	MQTT
Arquitectura	Cliente-servidor directo	Publicador-bróker-suscriptor
Persistencia	Sí	Sí
Escalabilidad	Media	Alta
Uso en IoT	Interfaces en tiempo real	Telemetría masiva

6️⃣ Simulación con CoAP

CoAP (Constrained Application Protocol):

- Diseñado para **dispositivos IoT limitados**
- Usa **UDP** en lugar de TCP
- Muy liviano

Qué observar en la simulación

- Mensajes más pequeños.
- Menor sobrecarga que HTTP.
- Funcionamiento tipo **REST**, pero optimizado.

7️⃣ Análisis conceptual por capas de red

Aunque aquí el foco es la simulación en Internet, debes poder explicar cada capa:

🧠 Capa 7 – Aplicación

Protocolos usados:

- **MQTT** → publicación/suscripción vía HiveMQ.
- **WebSocket** → comunicación bidireccional en tiempo real.
- **CoAP** → REST liviano sobre UDP.

Puntos a analizar:

- Formato del **payload**.

- Topics MQTT.
- Eventos en tiempo real.
- Diferencias de diseño.

📦 Capa 4 – Transporte

- **MQTT y WebSocket → TCP**
- **CoAP → UDP**

Puertos importantes:

- MQTT: **1883 / 8883**
- WebSocket HiveMQ: **8000**
- CoAP: **5683**

🌐 Capa 3 – Red

- Comunicación hacia **Internet pública**.
- IP destino → broker HiveMQ.
- Diferencia con Actividad 1 (localhost).

🔗 Capa 2 – Enlace

- Direcciones MAC de la red local.
- Gateway hacia Internet.
- No visible en cliente web, pero sí en capturas locales.

8️⃣ Qué demostrar en el informe

Tu entrega debería incluir:

✔ Funcionamiento

- FastAPI ejecutándose.
- Publicación MQTT hacia HiveMQ.
- Suscripción desde cliente web.
- Recepción de mensajes en Internet.

✔ Comparación de protocolos

- MQTT vs WebSocket vs CoAP.
- TCP vs UDP.
- Modelo de comunicación.

✔ Análisis por capas

- Aplicación
- Transporte
- Red
- Enlace

✔ Conclusión clave

El uso de un broker público permite visualizar datos IoT globalmente, demostrando:

- Escalabilidad de MQTT.
- Comunicación en tiempo real con WebSocket.

- Eficiencia de CoAP en dispositivos limitados.
